

BedLog Update Guide

Safe MacBook -> GitHub -> Raspberry Pi deployment workflow

Purpose: Use this procedure whenever you make BedLog changes on your MacBook and want to deploy them to the Raspberry Pi without accidentally sending macOS-only files or dependencies.

1. Important Rules

Rule	What to do
Do not commit node_modules	The Mac and Raspberry Pi need platform-specific Node packages. Let each machine install its own node_modules.
Do not use sudo npm ci	Run npm ci as user pi. Using sudo can create root-owned files and later cause Permission denied errors.
Keep package.json cross-platform	Do not explicitly add packages such as @rollup/rollup-darwin-arm64 or lightningcss-darwin-arm64.
Darwin entries in package-lock.json can be normal	Optional platform entries may exist in the lockfile. They are acceptable when marked optional and restricted to os: darwin.
Check git status before every commit/pull	This helps catch accidental files, nested repositories, local Pi changes, and generated files.

2. One-Time MacBook .gitignore Setup

In the BedLog project on your MacBook, make sure **.gitignore** contains at least the following:

```
# macOS
.DS_Store
.*
__MACOSX/
.AppleDouble
.LSOverride
.Spotlight-V100
.Trashes
.fsevents

# Node / Vite
node_modules/
dist/
.vite/
.cache/

# Logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Environment / secrets
.env
.env.local
.env.*.local

# Editors
```

```
.vscode/  
.idea/
```

Important: .gitignore does not remove files that Git already tracks. If unwanted files were previously committed, clean the Git index carefully before the next push.

3. Before Updating: Check the Mac Project

Open Terminal on the MacBook and go to your BedLog project directory. Confirm the branch and remote before making a release:

```
cd /path/to/bedlog  
git status  
git branch --show-current  
git remote -v  
  
grep -n "darwin" package.json  
grep -n "rollup-darwin\|lightningcss-darwin" package.json
```

4. Test the Update on the MacBook

Install dependencies and make sure the project builds before sending it to GitHub:

```
npm install  
npm run build
```

Never copy the Mac node_modules directory to the Raspberry Pi. macOS ARM64 and Linux ARM64 are different platforms.

5. Commit and Push from the MacBook

Inspect, stage, inspect again, then commit and push:

```
git status  
git add .  
git status  
git commit -m "Describe the BedLog update"  
git push origin main
```

Read the second git status carefully. Make sure node_modules, .DS_Store, __MACOSX, dist, secrets, and other unwanted files are not staged.

6. Update the Raspberry Pi

Use the live BedLog repository and check for local changes before pulling:

```
cd ~/Documents/bedlog
git status
```

If Git reports modified or unfamiliar untracked files, inspect them first. Do not immediately use `git add`, `git reset --hard`, or delete them.

If the working tree is clean:

```
git pull --ff-only origin main
git log -1 --oneline
```

7. Install Raspberry Pi Dependencies Correctly

Verify the Pi environment:

```
node --version
npm --version
node -p "process.platform + ' ' + process.arch"
```

Your proven working setup used Node v20.19.2, npm 10.9.9, and linux arm64. If npm is still version 9.x, upgrade it:

```
sudo npm install -g npm@10
hash -r
npm --version
```

Install project dependencies as the normal pi user:

```
npm ci
```

Do not run: `sudo npm ci`

8. Build and Restart BedLog

```
npm run build

sudo systemctl restart bedlog-backend
sudo systemctl restart bedlog-vite
```

Verify both services and application endpoints:

```
systemctl is-active bedlog-backend
systemctl is-active bedlog-vite
curl http://localhost:4002/health
curl -I http://localhost:5173
```

Expected backend: `{"ok":true}`. Expected frontend response includes **HTTP/1.1 200 OK**.

9. Final Git Check

```
cd ~/Documents/bedlog
git status
git log -1 --oneline
```

A successful deployment should show that main is up to date with origin/main and the working tree is clean.

10. Troubleshooting

A. EBADPLATFORM / rollup-darwin-arm64

First confirm that a Darwin-only package is not explicitly declared in package.json:

```
grep -n "darwin" package.json
grep -n "rollup-darwin\\|lightningcss-darwin" package.json
```

Optional Darwin entries in package-lock.json are normal when they are marked optional and restricted to os: darwin. In this BedLog deployment, upgrading the Pi from npm 9.2.0 to npm 10.9.9 allowed npm ci to install correctly on Linux ARM64.

B. Permission denied while removing node_modules

This usually means some files were created as root. Remove the old directory once with elevated permission, then reinstall normally:

```
cd ~/Documents/bedlog
sudo rm -rf node_modules
npm ci
```

C. Embedded Git repository warning

Do not keep another cloned Git repository inside ~/Documents/bedlog. The live service paths are:

```
Frontend: /home/pi/Documents/bedlog
Backend: /home/pi/Documents/bedlog/db-backend
```

D. npm audit warning

Do not automatically run **npm audit fix --force** on the production Pi. It may introduce breaking dependency changes. Review and test dependency changes separately.

11. Quick Routine for Every Future Update

MacBook

```
cd /path/to/bedlog
git status
npm install
npm run build
git add .
git status
git commit -m "BedLog update"
git push origin main
```

Raspberry Pi

```
cd ~/Documents/bedlog
git status
git pull --ff-only origin main
npm ci
npm run build
sudo systemctl restart bedlog-backend bedlog-vite
systemctl is-active bedlog-backend bedlog-vite
curl http://localhost:4002/health
curl -I http://localhost:5173
git status
```

Golden rule: GitHub stores the project source and lockfile; the MacBook and Raspberry Pi each create their own node_modules locally. This is the key to avoiding macOS-versus-Linux dependency errors.